МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

ПРОМЕЖУТОЧНЫЙ ОТЧЕТ № 2

Учебная практика Аналитический этап
Вариант 25 Учет посещаемости

Студент группы ИКБО-67-24

«03» ноября 2026 г. Степанов А.Д.
(подпись и расшифровка подписи)

Руководитель практики
от кафедры

«___» _____ 2026 г. Образцов В.М.
(подпись и расшифровка подписи)

Москва 2026 г.

1 СХЕМА БАЗЫ ДАННЫХ

На аналитическом этапе реализована серверная часть системы учета посещаемости студентов: база данных, запуск через Docker, вход с разграничением ролей, операции с данными и API. Предметная модель включает семь связанных таблиц. Основной объект учета — отметка конкретного студента на конкретном занятии.

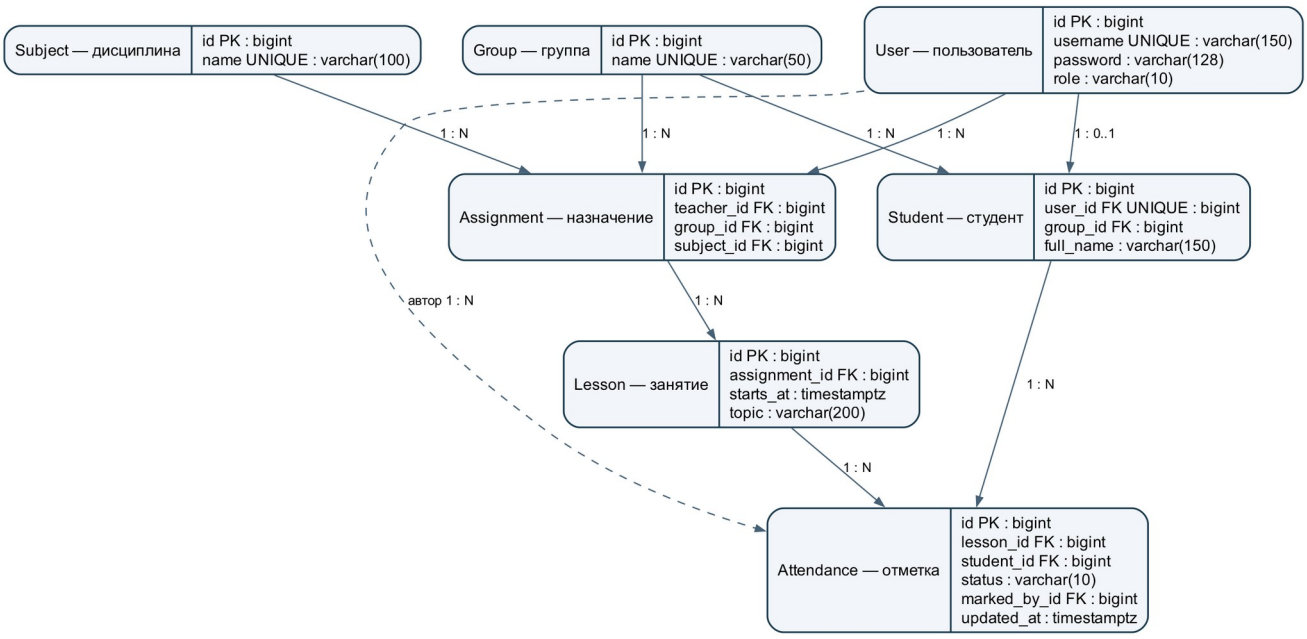


Рисунок 1.1 — Предметная схема базы данных

PK обозначает первичный ключ, FK — внешний ключ, UNIQUE — запрет повторов. В PostgreSQL имена предметных таблиц начинаются с journal_. Идентификаторы имеют тип bigint, время хранится с часовым поясом. Стандартные таблицы Django для сессий и разрешений на схеме не показаны.

Уникальны логин пользователя, название группы, название дисциплины, профиль пользователя, тройка «преподаватель — группа — дисциплина», пара «назначение — время занятия» и пара «занятие — студент». Для статуса отметки задано ограничение CHECK: present, absent или excused. Миграция и соответствующий SQL включены в проект.

2 ЗАПУСК ЧЕРЕЗ DOCKER

Настроены два сервиса Docker Compose: web с Django и Gunicorn и db с PostgreSQL 17; данные сохраняются в отдельном томе, а сервер запускается после проверки готовности БД. При первой проверке возникла ошибка «Cannot connect to the Docker daemon», поскольку Docker Desktop не был запущен; после запуска приложения сборка и запуск контейнеров завершились успешно. Оба сервиса получили состояние healthy, приложение доступно по адресу http://localhost:8000, а PostgreSQL доступен только внутри сети Compose.

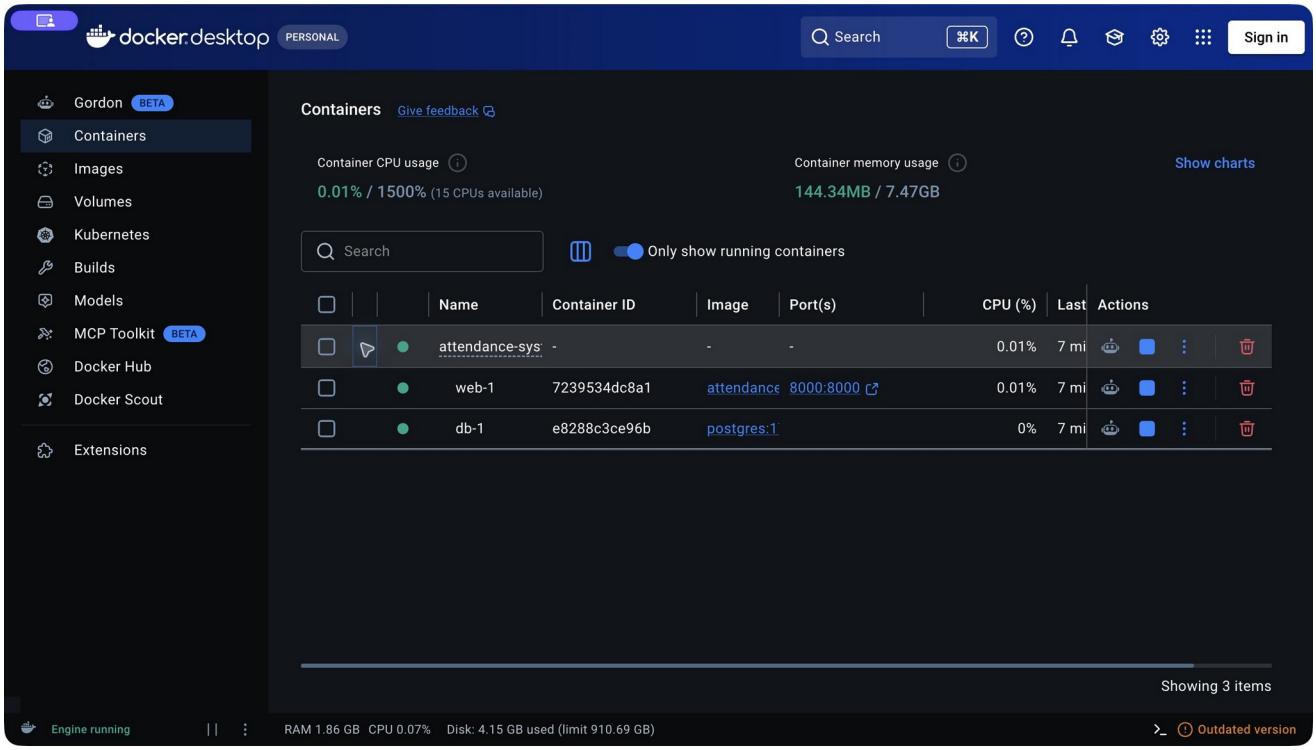


Рисунок 2.1 — Запущенные контейнеры web и db в Docker Desktop

Команда запуска: docker compose up -d --build. При старте автоматически применяются миграции. Параметры подключения и секретный ключ вынесены в локальный файл .env, который исключен из Git; для повторного запуска на другом компьютере подготовлен .env.example.

Демонстрационный набор создается отдельной командой seed_demo и содержит вымышленных студентов. Проверка /health/ выполняет запрос к базе данных, поэтому статус web подтверждает доступность не только HTTP-сервера, но и подключения к PostgreSQL.

3 АУТЕНТИФИКАЦИЯ

Для входа использована сессионная аутентификация Django. Пользователь вводит логин и пароль, а сервер сверяет пароль с сохраненным хешем, поэтому открытый пароль в базе не хранится. После успешной проверки Django создает сессию и передает браузеру cookie с ее идентификатором. В следующих запросах браузер отправляет эту cookie, и сервер определяет текущего пользователя и его роль. Самостоятельная регистрация создает только студента; преподавателя и администратора может создать администратор через API. Для запросов, изменяющих данные, проверяется CSRF-токен, а выход из системы завершает сессию. Права проверяются на сервере: студент видит свои отметки, преподаватель работает со своими назначениями, администратор управляет справочниками.

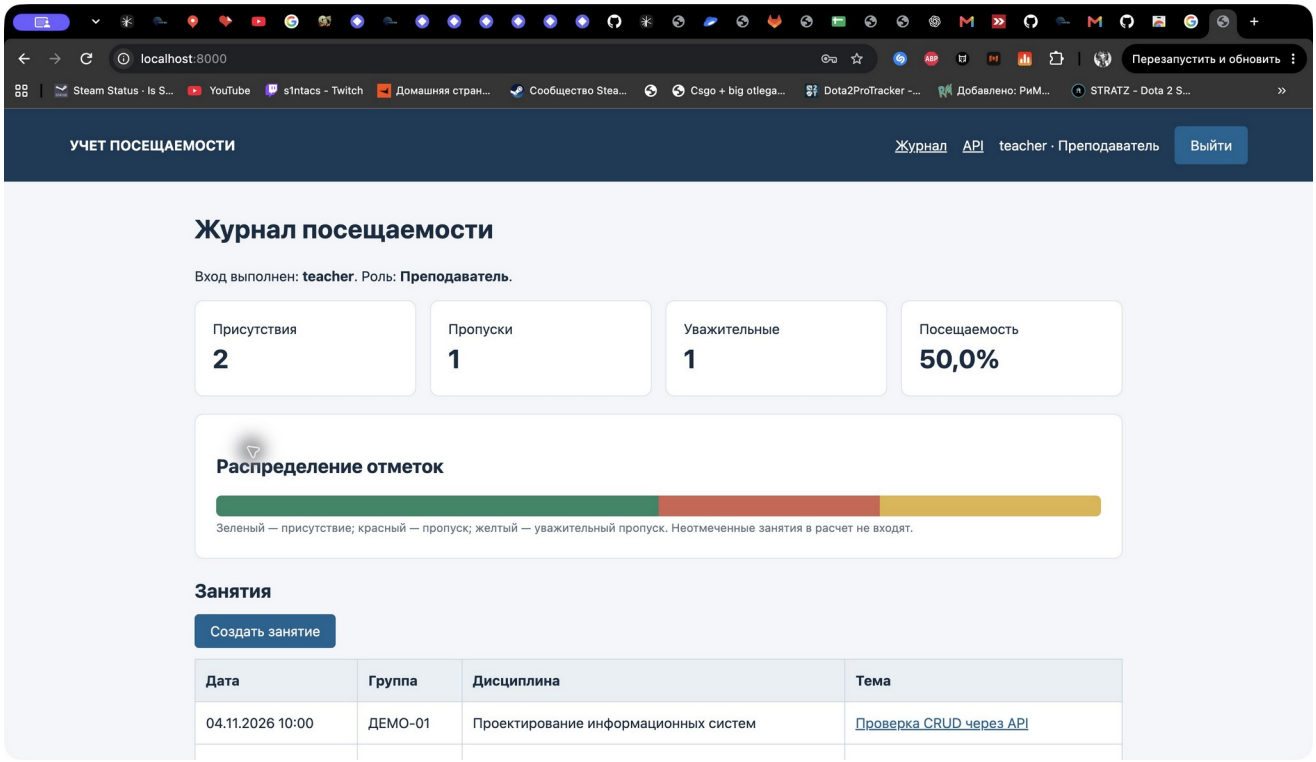


Рисунок 3.1 — Успешный вход пользователя teacher с ролью преподавателя

4 ОСНОВНЫЕ ОПЕРАЦИИ С ДАННЫМИ

Таблица 4.1 — Реализованные операции CRUD

Объект	**Создание и чтение**	**Обновление и удаление**
Пользователи	Создание учетной записи, список и просмотр — администратор	Изменение данных и роли; удаление при отсутствии защищенных связей
Группы, студенты, дисциплины, назначения	POST создает, GET возвращает доступные записи	PUT/PATCH изменяют; DELETE защищает связанные записи
Занятия	Преподаватель создает и просматривает свои занятия	Изменение даты и темы; удаление с отметками требует подтверждения
Посещаемость	Создание и чтение отметок в пределах прав роли	PATCH меняет статус; DELETE удаляет отметку

Пример ниже выполнен встроенным HTTP-клиентом на странице /api/docs/: отправлен POST /api/lessons/ с назначением, временем и темой. Ответ 201 Created и идентификатор 4 подтверждают создание записи в PostgreSQL.

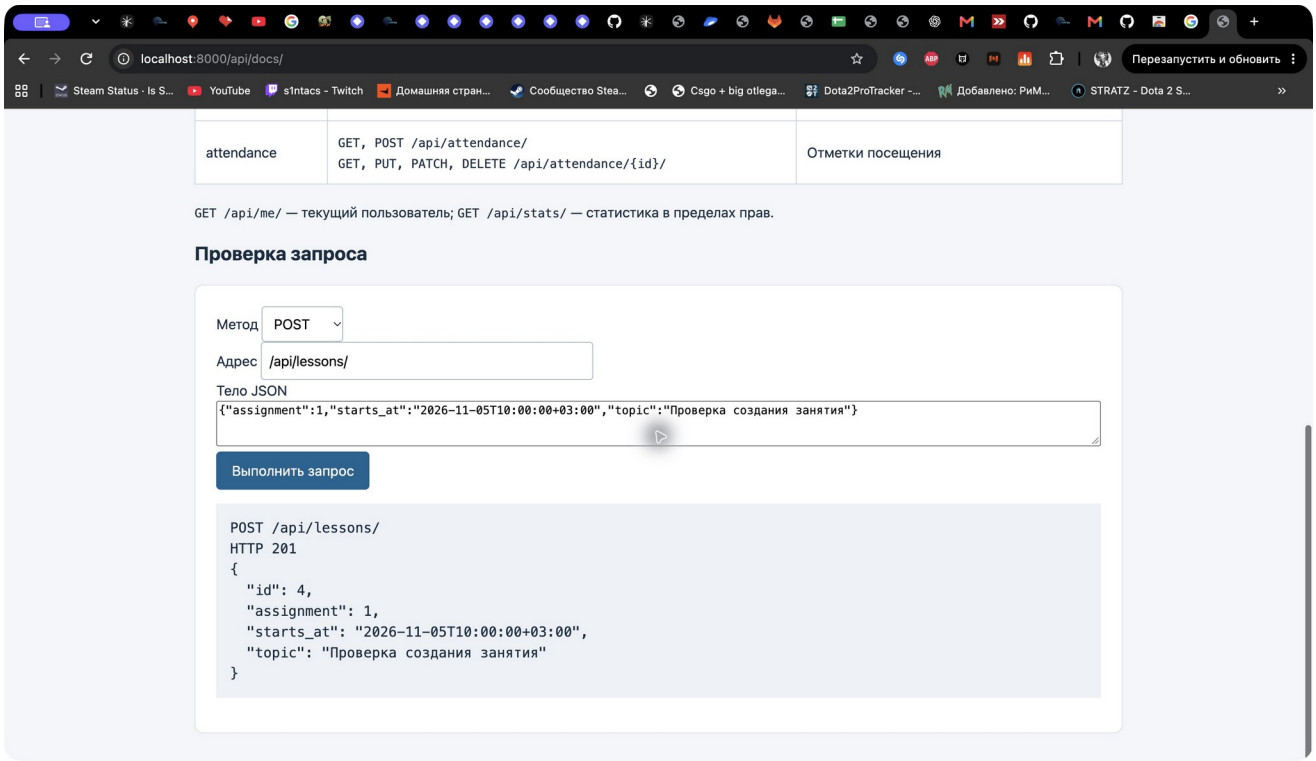


Рисунок 4.1 — Запрос создания занятия и фактический ответ HTTP 201

5 СВЯЗИ И БИЗНЕС ЛОГИКА

В одной учебной группе может быть много студентов, но профиль студента относится к одной группе. Учетная запись связывается с профилем студента отношением один к одному. После самостоятельной регистрации профиль и группу назначает администратор.

Назначение объединяет преподавателя, группу и дисциплину. Одному назначению соответствует много занятий с разными датами и темами. Преподаватель получает доступ только к занятиям своих назначений, поэтому прямой запрос чужого идентификатора не открывает чужой журнал.

У занятия может быть много отметок, а у студента — много отметок по разным занятиям. Каждая отметка хранит статус, автора и время последнего изменения. Пара «занятие — студент» уникальна: повторное сохранение формы обновляет запись, а повторное создание через API отклоняется; для изменения используется PATCH.

Перед сохранением сервер проверяет, что студент входит в группу занятия и статус допустим. После появления отметок нельзя менять группу студента или назначение занятия; назначение с занятиями также нельзя переназначить другому преподавателю. Эти проверки защищают согласованность уже заполненного журнала.

Справочники, на которые ссылаются другие записи, защищены от удаления. Удаление занятия вместе с отметками требует отдельного подтверждения и выполняется в транзакции: либо удаляются все связанные отметки и занятие, либо изменения откатываются.

Посещаемость рассчитывается как число присутствий, деленное на число всех установленных отметок, умноженное на 100 процентов. Уважительный пропуск учитывается отдельно, но входит в общее число отметок. Неотмеченное занятие не считается пропуском и не входит в знаменатель; при отсутствии отметок выводится «Нет данных». В демонстрации две отметки присутствия из четырех дают 50 процентов.

6 АДРЕСА API

Для каждого ресурса реализованы список, создание, просмотр одной записи, полное и частичное обновление и удаление. В таблице {id} означает числовой идентификатор; доступ к данным ограничивается ролью и назначениями пользователя.

Таблица 6.1 — Адреса ресурсов API

Ресурс	**Список и создание**	**Одна запись**
Пользователи	GET, POST /api/users/	GET, PUT, PATCH, DELETE /api/users/{id}/
Группы	GET, POST /api/groups/	GET, PUT, PATCH, DELETE /api/groups/{id}/
Студенты	GET, POST /api/students/	GET, PUT, PATCH, DELETE /api/students/{id}/
Дисциплины	GET, POST /api/subjects/	GET, PUT, PATCH, DELETE /api/subjects/{id}/
Назначения	GET, POST /api/assignments/	GET, PUT, PATCH, DELETE /api/assignments/{id}/
Занятия	GET, POST /api/lessons/	GET, PUT, PATCH, DELETE /api/lessons/{id}/
Отметки	GET, POST /api/attendance/	GET, PUT, PATCH, DELETE /api/attendance/{id}/

GET — получить данные; POST — создать; PUT — полностью обновить; PATCH — изменить отдельные поля; DELETE — удалить. Удаление занятия с отметками выполняется по адресу /api/lessons/{id}/?confirm=true. Без подтверждения сервер возвращает 409.

Дополнительные адреса: GET /api/me/ — текущий пользователь; GET /api/stats/ — статистика; GET /api/ — список ресурсов; GET /api/docs/ — справочник и инструмент запросов; GET /health/ — проверка сервера и БД.

Вход, регистрация и выход выполняются HTML-формами POST /login/, POST /register/ и POST /logout/. Для изменения данных API используются cookie сессии и заголовок X-CSRFToken. Ошибки валидации возвращают 400, отсутствие доступа — 403, недоступный объект — 404, конфликт связанных данных — 409.

7 ПРОВЕРКА API В БРАУЗЕРЕ

Справочник API доступен после входа в систему и содержит адреса всех предметных ресурсов. Ниже показан настоящий экран приложения; в нижней части страницы находится форма отправки HTTP-запросов, использованная для проверки CRUD.

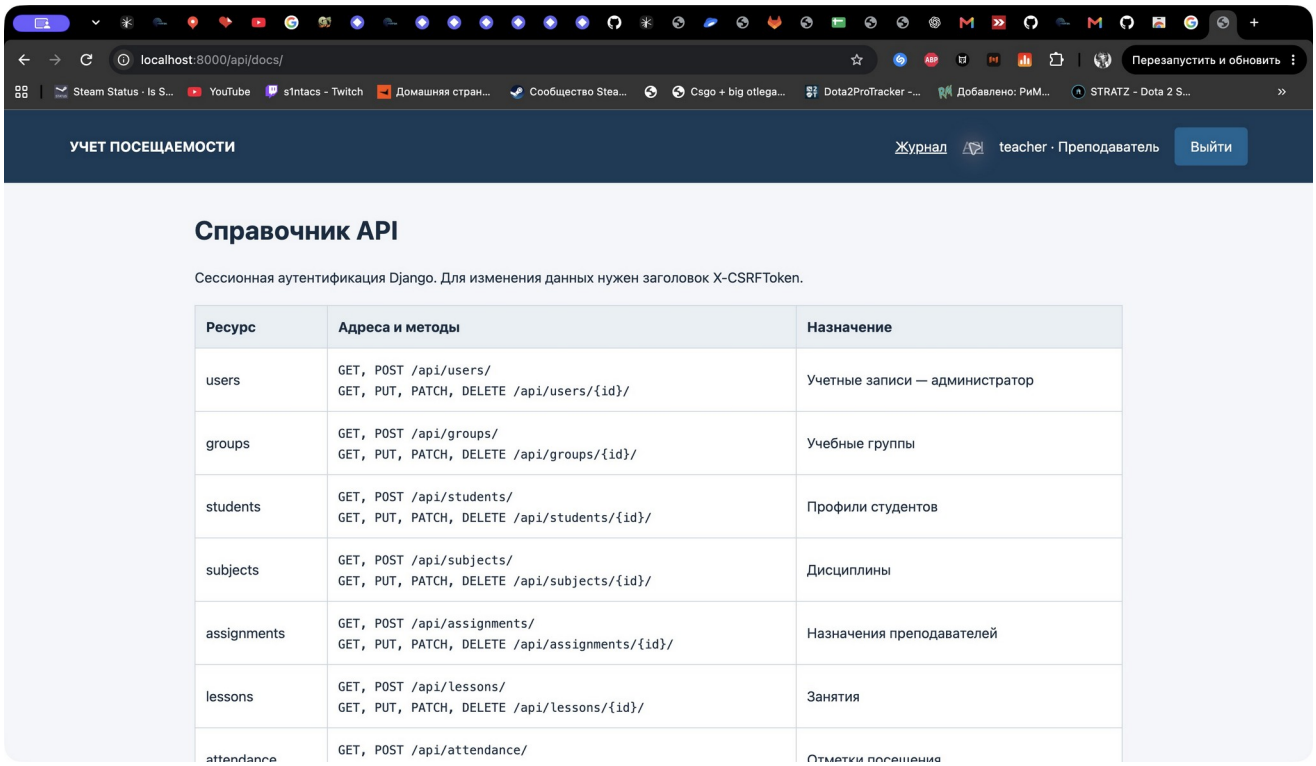


Рисунок 7.1 — Справочник адресов API в работающем приложении

Полный перечень методов и форматов полей сохранен в docs/api/endpoints.md, каталог OpenAPI — в docs/api/openapi.json. При проверке запрос на создание занятия выполнен от преподавателя и использует его собственное назначение. Изменяющие запросы студента и запросы преподавателя к чужим занятиям отклоняются; эти случаи дополнительно проверены тестами.

8 САМОПРОВЕРКА И CODE REVIEW

На момент подготовки отчета внешние замечания руководителя или напарника не получены. В ходе собственной проверки уточнены ограничения целостности и права доступа. Основные решения приведены в таблице 8.1; их поведение закреплено автоматическими тестами.

Таблица 8.1 — Решения по результатам собственной проверки

Риск или замечание	**Что исправлено или уточнено**
Дубли отметок	Уникальность пары занятие–студент и обновление записи при повторном сохранении формы.
Доступ к чужому журналу	Фильтрация списков и отдельных объектов по роли и назначениям.
Нарушение истории при смене связей	Запрет изменения группы после отметок, назначения после занятий и связанной роли пользователя.
Случайное удаление отметок	Подтверждение удаления занятия, транзакция и защита связанных справочников.
Неверная статистика	Неотмеченные занятия исключены из расчета, пустая выборка возвращает «Нет данных».
Поддельный запрос изменения	CSRF-защита формы входа и API с сессионной аутентификацией.

Все 26 автоматических тестов успешно выполнены на PostgreSQL и SQLite. Дополнительно проверены согласованность миграций, запуск двух контейнеров, вход преподавателя и создание занятия через API. Для последующих изменений настроен запуск тестов в GitHub Actions.

Для внешнего Code-Review подготовлены описание Pull Request и инструкция для Kiromaka. После публикации изменений ревьюер должен проверить код и отправить Approve либо замечания через Request changes. Получение одобрения и объединение ветки на этом этапе не заявляются.

9 ВЫВОД

На аналитическом этапе спроектирована и реализована база данных системы учета посещаемости. Созданы семь предметных таблиц и связи между ними, определены уникальные ключи и правила сохранения данных. Приложение и PostgreSQL запущены через Docker Compose; работоспособность подтверждена проверками контейнеров и запросами к серверу.

Реализованы регистрация и вход, три роли, CRUD справочников, занятий и отметок, журнал и расчет посещаемости. Подготовлены API, документация и инструмент проверки запросов. Все 26 автоматических тестов прошли, включая сценарии запрета чужих данных, защиты от дублей и подтверждения удаления.

Следующий шаг — публикация изменений в GitHub и внешнее ревью Kiromaka с фиксацией результата в Pull Request. Экспорт CSV, история аудита и расширенные фильтры остаются задачами следующих этапов. Подготовленная реализация позволяет демонстрировать основные сценарии на вымышленных данных и служит основой для дальнейшего развития проекта.